



NYU

ZK-Solvency

Professor: Benedict Bunz
PRESENTED BY Vincent Xu
05/04/2026

Problem Definition

A centralized exchange holds users' funds. How can we tell they are solvent?

- Tradeoff between **Privacy** and **Transparency**
 - A published balance sheet can be faked
 - Full disclosure destroys privacy

Can cryptography make solvency a verifiable fact rather than just a trusted claim?



Problem Elaboration

Can we prove **assets** $\geq$ **liabilities** without revealing balance?

A naive answer: publish a **ZK proof** that assets $\geq$ liabilities

However, the exchange controls **which accounts go into the proof**.

A dishonest exchange could exclude the accounts it owes most:

Exchange owes User A \$2M, leave user A out of the proof, it could still pass.

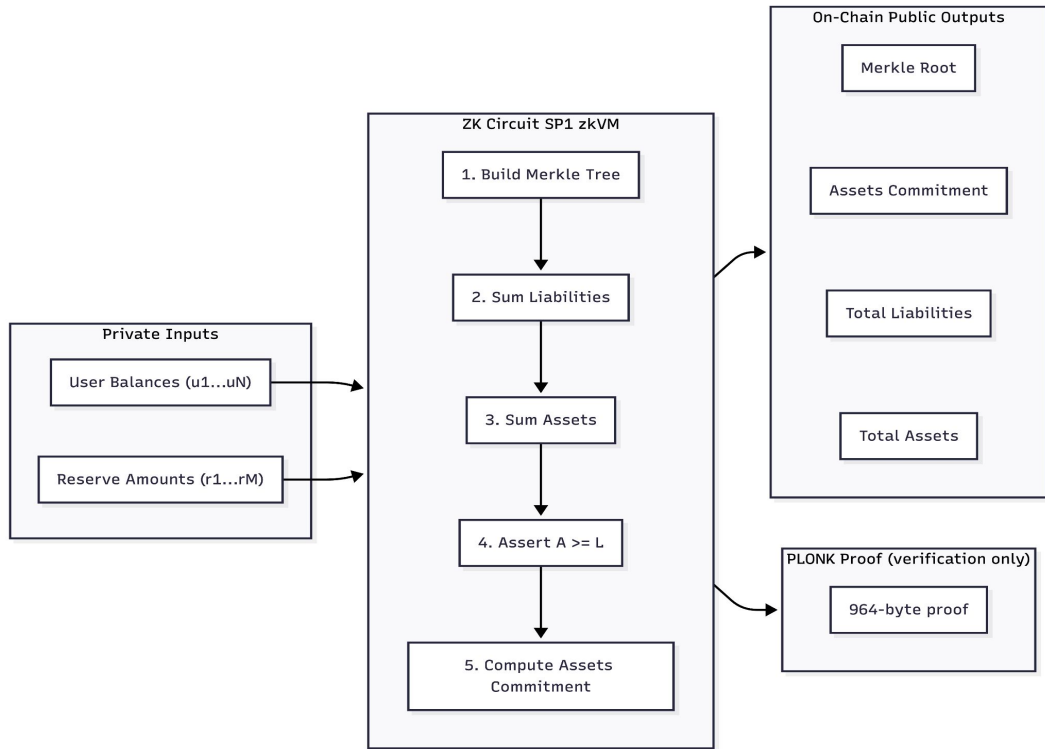


Merkle Tree Inclusion Proof

Bind the full account set in a Merkle root — users verify their account is committed, cryptographically.

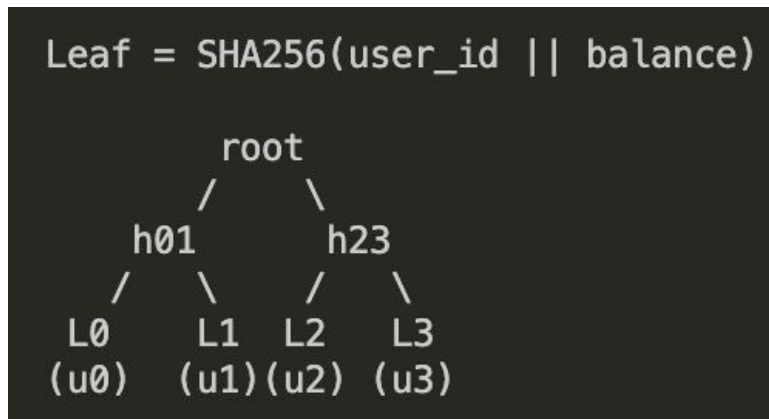
Component 1: ZK Proof

- Soundness: if assets < liabilities, the assertion at step 4 fails — no valid proof can exist. The exchange cannot lie.
- Zero-knowledge: the proof reveals nothing about individual balances, only the four public outputs



Component 2: Merkle Tree

- Every user's balance is committed as a leaf. The root is a 32-byte fingerprint of the entire liability set.
- User knows their ID for generating **$O(\log N)$** sibling path and checking path hashes up to the published root.

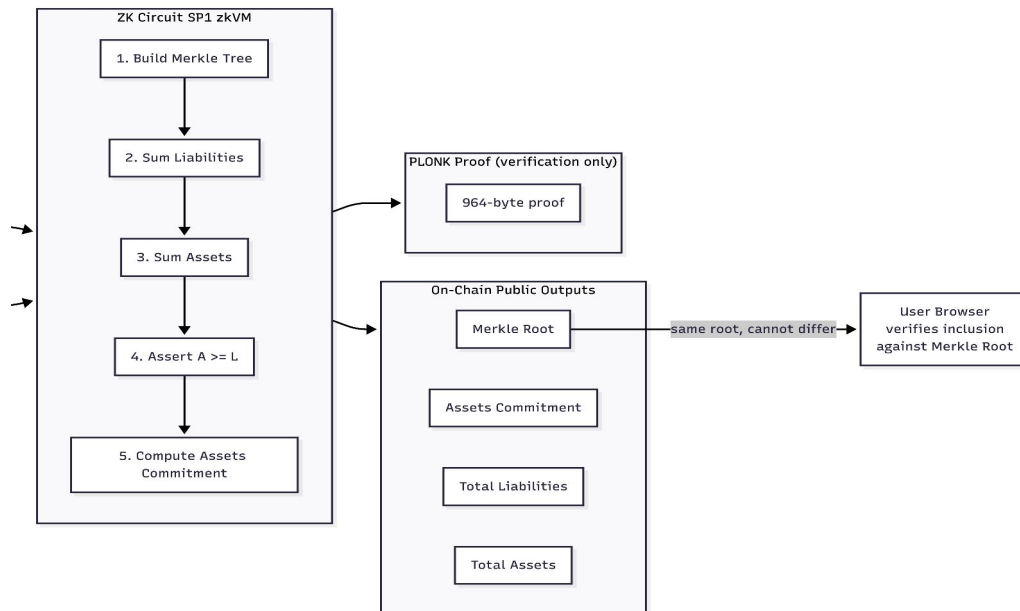


- **What it proves:** It proves a user's balance is committed in the exact tree used to compute total liabilities in the ZK proof
- **Where it runs:** The inclusion program runs entirely in the browser via Web Crypto API: no need to trust any server or binary

The Binding: One Proof, One Tree

How do we stop the exchange from using one set of accounts for the ZK proof and showing users a different Merkle tree to verify against?

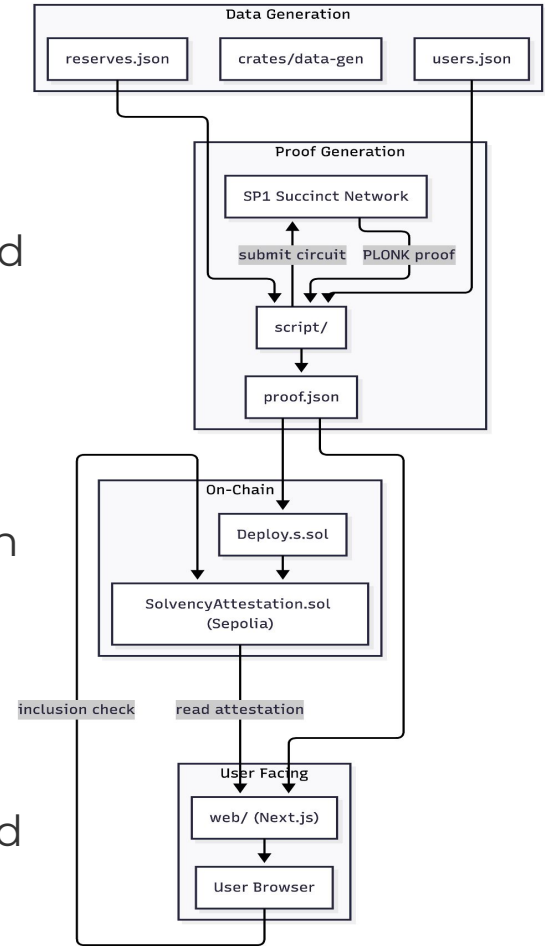
The shared root is not computed by the exchange and handed to us, it's **computed inside the proof circuit**, where we can **verify** it.



- Cannot substitute: one root, on-chain, produced by the proof — no second root possible.
- Cannot exclude silently: removing any account changes the root — the user's inclusion check fails.

System Architecture

- **Data Gen:** Simulating user and reserve accounts
- **Proof Gen:** Call SP1 network to run proof on Succinct and get PLONK proof and public outputs
 - **Assets commitment:** order-sensitive hash over all reserves — prevents fake reserve sets that sum
- **On-Chain:** The public outputs (merkle root, assets commitment, liabilities, assets, timestamp) are stored on Sepolia, a Ethereum test network
- **User Interface:** Display attestation data and provide inclusion check runs entirely in browser
 - **Browser-side verification:** inclusion check runs in browser via Web Crypto API, no server trust required



Benchmarks

Merkle Operations Scale as $O(\log N)$

N	Build	Prove	Verify
100	0.09ms	0.14 μ s	3.1 μ s
5000	3.1ms	0.15 μ s	3.8 μ s

- Prove and verify barely move as N grows 50x.
- Build scales linearly - dominated by N SHA-256 hashes.

ZK proof generation scales linearly with N

N	Time
100	0.5 s
5000	27s

- More users means more SHA-256 calls inside the circuit $\rightarrow$ more constraints
- Note, we are just measuring the time taken to execute the Rust program as a RISC-V trace (the circuit execution time, not the cryptographic proving time)
- On-chain verification: ~250k gas, constant regardless of N

Demo

Takeaways

- ZK + Merkle is a pattern, not just a tool
 - Aggregate claim (ZK) + membership proof (Merkle) + shared binding = verifiable solvency. Generalizes to any system with claims over individual records.
- The cryptography is complete; the operational layer is not
 - Proof is sound, contract is live, inclusion checker works
 - Missing: automated attestation, exchange identity binding, authenticated lookup

Thanks for watching!